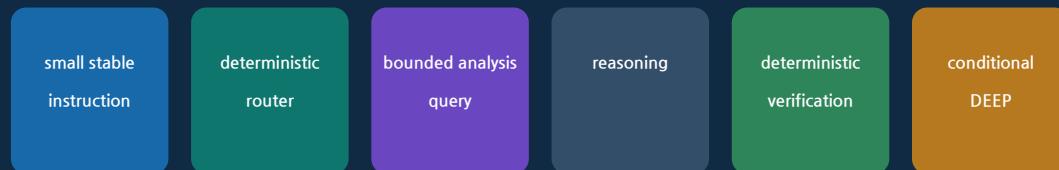


CTF-Rat v2

업그레이드 및 아키텍처 설계서

Query-First Solver Runtime

Sol 운영 전략 / 모델 tier-reasoning 설정은 범위에서 제외



Source snapshot

main: 01a1a9a1a73eb0b8f4c5a213662aec027d1fadb3

dev : a6e6720068f61d3c1f2398804061682079fcbf64

문서 통제

항목	내용
문서 목적	CTF-Rat v2의 업그레이드 및 아키텍처 변경을 구현 가능한 설계 수준으로 정의한다.
범위	저장소 구조, runtime/orchestration, query interface, cache/artifact, state projection, REV acceleration, telemetry/benchmark, CI/test, compatibility/migration.
명시적 제외	Sol 운영 전략, 모델 tier 선택, reasoning effort 설정, 모델별 라우팅 정책. 모델에 종속되지 않는 context layout과 prompt-prefix 안정화 원칙은 아키텍처 항목으로 포함.
근거	업로드 분석 3종 + CTF-Rat 실제 main/dev 소스 및 CI/benchmark 스키마 확인.
상태	PROPOSED DESIGN. 구현·benchmark 완료를 의미하지 않는다.

표기 규칙

표기	의미
VERIFIED	실제 저장소 소스/스키마/CI에서 확인한 사실
INFERRED	확인된 사실에서 도출한 구조적 분석
PROPOSED	구현 또는 정책 변경 제안
MEASURED	재현 가능한 benchmark로 수치가 확인된 상태; 본 설계서에는 현재 해당 수치 없음
BLOCKED	근거 또는 구현/측정이 부족해 완료로 선언할 수 없는 항목

구현 분류	정의
EXISTING	이미 구현되어 재사용해야 함
PARTIAL	구조/기능 일부가 있으나 v2 contract에 미달
DUPLICATE	같은 개념이 둘 이상의 경로/캐시/인터페이스로 존재
GAP	현재 소스에서 확인된 실제 공백
REJECT	v2.0에서는 추가하지 않거나 명시적으로 금지

목차

문서 통제	2
표기 규칙	2
1. 결론	6
1.1 최상위 불변식	6
2. 범위와 비범위	6
3. Source First — 현재 구현 근거	7
4. AS-IS 구성과 문제 분류	7
4.1 AS-IS 실행 비용 구조	8
5. v2 아키텍처 원칙	9
6. TO-BE — Query-First Solver Runtime	9
6.1 구성 요소	10
6.2 Context layout	10
7. P0-0 — Measurement First	10
7.1 Benchmark result v2 필수 필드	11
7.2 `duplicate_tool_calls` 정의	11
7.3 변경 파일	11
7.4 Acceptance	11
8. P0-1/P0-2 — Slim Agent Runtime + FAST/DEEP	11
8.1 Root contract에 남길 7개 규칙	12
8.2 FAST에서 기본 비활성	12
8.3 Progress Novelty Governor	12
9. P0-3 — Canonical Cache / Artifact	12
9.1 Canonical key	12
9.2 Migration model	13
9.3 Cache hit envelope fix	13
9.4 자동 재사용 허용/금지	13
10. P0-4 — Context Projection: `state compact` 강화	13
10.1 우선순위 projection	14
10.2 Context budget semantics	14

11. P1 — Single Front Door: `rat`	14
11.1 `rat route` contract	14
11.2 Router invariants	14
12. P1 — Operator Skill Layer	15
13. P1 — Function Card v2	15
13.1 Schema rule	15
14. P1 — Oracle Detector → Symbolic → Concrete Verify	16
14.1 자동 연결 조건	16
15. P1/P2 — Bounded Backward Data Slice	16
15.1 MVP 범위	16
15.2 결과 계약	17
16. P2 / v2.1 — Benchmark-Driven Extensions	17
17. 핵심 인터페이스 / 스키마 설계	18
17.1 Query result envelope	18
17.2 Output budget contract	18
17.3 Error taxonomy	18
18. 구현 계획 — PR 단위	18
19. 테스트 / 회귀 설계	19
19.1 Unit tests	19
19.2 Integration / E2E	19
19.3 Failure-path tests	19
20. Benchmark / Ablation 방법	20
20.1 권장 ablation	20
20.2 실행 규칙	20
21. v2 Release Gate	20
22. Compatibility / Migration / Rollback	21
22.1 Canonicalization 단계	21
22.2 Rollback	21
23. 위험 분석	21
24. 보안 및 검증 경계	21

25. P0 / P1 / P2 / Reject	22
26. Definition of Done	22
부록 A. 파일 변경 맵	23
부록 B. Architecture Decision Records	23
부록 C. 예시 운영 흐름	24
C.1 단순 REV checker — FAST	24
C.2 불확실한 REV — DEEP escalation	24
C.3 Warm cache	24
부록 D. Source Reference	25
최종 요약	25

1. 결론

결론

CTF-Rat v2의 주된 변화는 기능 추가가 아니라 실행 경로 단순화다. 이미 존재하는 deterministic 분석 도구를 하나의 query/caching/verification 경로로 연결하고, FAST를 기본값으로 하며 불확실할 때만 DEEP으로 승격한다.

VERIFIED: `revq`, Ghidra `decomp`, typed STATE, artifact/cache, `rat-profile`, `rat-slice`, `rat-dyn`, `rat-verify`, symbolic solve/concrete verification을 포함한 상당한 도구 기반이 이미 존재한다. `CLAUDE.md`는 시작 시 여러 doctrine/grounding 문서를 읽고 Scout/Hypothesis/Primitive/Adversarial Verify까지 기본 workflow로 노출한다. [R1-R8]

INFERRED: v2의 1차 병목은 분석 능력 부족보다 orchestration tax, context bloat, 캐시 세계 분산, raw output 재주입, 고정적인 subagent/skeptic 정책에 가깝다. 이 판단은 업로드된 세 분석 문서와 저장소 구조가 같은 방향을 가리킨다. [U1-U3]

PROPOSED: 목표 구조를 아래로 고정한다.

CTF-Rat v2 canonical flow

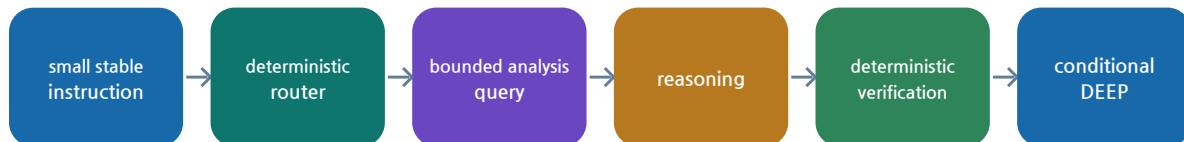


그림 1. v2 canonical runtime. 모델/에이전트가 저장소 전체를 따라가는 대신, deterministic 분석 산출물에 bounded query를 수행한다.

핵심 구현 순서: `measure → slim → unify cache → compact context → query front door → oracle wiring → bounded data slice`. 이 순서를 깨고 신규 decompiler/cache/state DB/full taint/generic auto-exploit부터 추가하지 않는다.

1.1 최상위 불변식

- Correctness first: FAST는 검증 생략 모드가 아니다. `SOLVED`, `PASS`, `VERIFIED`, `FIXED`에는 executable oracle, unit/integration test, schema validation, benchmark 등 실제 증거가 필요하다.
- One concept → one canonical implementation: revq/decomp/ratlib가 같은 정보를 별도 캐시·스키마·경로로 장기 유지하지 않는다.
- Repository/artifact DB = long-term truth: 모델 context는 현재 working set만 보유한다.
- Measurement before architecture claims: 속도/토큰/solve-rate 개선은 동일 corpus·환경·timeout에서 전후 수치로 증명한다.
- Deterministic fact와 interpretation 분리: LLM semantic summary는 자동 캐시 주입 대상이 아니다.

2. 범위와 비범위

이 설계서는 사용자가 요청한 대로 Sol 운영 전략을 제외한다. 따라서 특정 모델 이름, 모델 tier, reasoning effort, 모델별 fallback/routing, 가격·latency 비교는 다루지 않는다. 다만 모델과 무관하게 runtime 비용을 줄이는 stable prefix, bounded query, lazy skill, context projection은 아키텍처에 포함한다.

범위	포함 여부	설계 기준
Agent/runtime endpoint	포함	작고 안정된 instruction, FAST/DEEP, lazy skill
Router & query API	포함	deterministic route + one front door

범위	포함 여부	설계 기준
Cache & artifact	포함	ratlib canonical + provenance + cold/warm telemetry
STATE	포함	append-only truth 유지 + bounded compact projection
REV acceleration	포함	Function Card v2, oracle wiring, backward data slice MVP
PWN extensions	조건부	benchmark 병목이 증명될 때만 observer/collector 추가
Benchmark/CI	포함	telemetry schema, ablation, regression gate
Model/Sol operation	제외	모델 tier/effort/routing 전략 없음
Third-party target automation	제외	로컬 fixture/challenge artifact/Docker/loopback 중심

3. Source First — 현재 구현 근거

저장소 확인 snapshot은 `main` 01a1a9a... / `dev` a6e6720... 기준이다. 아래는 설계 판단에 직접 사용한 근거다.

ID	경로	확인 내용	상태
R1	main/CLAUDE.md	START HERE에서 SOLVING→SOLVABILITY→PRIMITIVE_GATE→GROUNDING_INDEX 읽기 순서를 요구하고, 0~7 단계 workflow·Scout·가설 fan-out·primitive gate·adversarial verify를 기본 경로로 노출.	VERIFIED
R2	main/bin/revq	큰 정적 읽기를 context 밖으로 빼고 한 번 추출·캐시한 뒤 slice만 회수하도록 설계. `--func`, string↔function xref, interesting heuristic, evasion signal 존재. sidecar `revq.json` cache 사용.	VERIFIED
R3	main/bin/ratlib/cache.py	tool/inputs/parameters/dependencies/policy_digest/output_schema를 canonical JSON으로 해시하는 content-addressed cache index 존재.	VERIFIED
R4	main/bin/ratlib/contracts.py	기존 deterministic tool을 tool-result envelope/artifact/cache에 adapter. 성공·비truncate 결과만 재사용. cache hit 시 저장된 envelope를 그대로 반환.	VERIFIED
R5	main/bin/ratlib/analysis.py	P2 bounded analysis executors. `rat-slice` limitation이 "call-path reachability only; no value-flow proof". envelope의 cache key=None/hit=False.	VERIFIED
R6	main/bin/state	typed STATE + `delta`, `compact`, `checkpoint`가 이미 존재. compact는 observations/findings/primitives/hypotheses/ruled_out/unknowns/n_ext_probes materialized view 출력. budget/text priority 없음.	VERIFIED
R7	main/bin/decomp	Ghidra 전체 decompile 1회 후 함수별 조회. 별도 `decomp` cache와 provenance validation 사용.	VERIFIED
R8	main/bin/ratlib/decomp_cache.py	binary SHA-256, Ghidra version, script hashes, analyzer options를 provenance key에 포함.	VERIFIED
R9	main/.github/workflows/regression.yml	unittest, e2e artifact handoff/analysis/orchestration, pkselftest를 CI에서 실행.	VERIFIED
D1	dev/benchmarks/schemas/rat.benchmark-result.v1.json	현재 result schema는 사실상 `schema`만 required이고 additionalProperties=true.	VERIFIED
D2	dev/benchmarks/ablations/A0..A5.yaml	A0~A5 ablation 뼈대 존재. A5에 legacy-bin/state-v2/artifact-store/slice/dyn/verify/p2-fanout/context-governor 및 cold/warm attempt 정의.	VERIFIED

업로드 문서 U1/U2/U3는 위 소스 근거를 종합해 architecture 방향, 우선순위, 신규 기능 gate를 제안한다. 설계서에서는 세 문서의 공통분모를 채택하되, 실제 소스가 이미 제공하는 기능은 신규 subsystem으로 재작성하지 않는다.

4. AS-IS 구성과 문제 분류

현재 CTF-Rat는 “도구 부족” 상태가 아니다. 문제는 같은 목적을 수행하는 경로가 여러 층으로 노출되고, agent 진입점에서 그 절차를 넓게 읽도록 요구한다는 점이다.

영역	현재 상태	분류	v2 처리
Agent entry	CLAUDE/AGENTS가 doctrine/grounding 및 다단계 workflow를 기본 노출	EXISTING + OVERWEIGHT	root contract 축소, 세부 doctrine lazy
REV static	revq가 functions/strings/xrefs/interesting/evasion + sidecar cache 제공	EXISTING	Function Card v2와 canonical cache로 승격
Decompile	Ghidra provenance cache + 함수별 조회	EXISTING	cache index 통합, artifact reference화
Cache	ratlib cache + revq sidecar + decomp sidecar	DUPLICATE	ratlib cache/artifact를 canonical index로
State	typed event stream + compact/delta/checkpoint	PARTIAL	compact에 max-bytes/format/priority projection 추가
Analysis envelope	rat-*가 artifact/result envelope를 생성하지만 cache 연결 없음	PARTIAL	canonical cache 연결 및 provenance 통일
Slice	CFGFast callgraph + VEX block metadata	PARTIAL	bounded backward data slice MVP
Routing	profile/revq가 signals/routes/interesting 후보를 제공	PARTIAL	`rat route` deterministic contract로 통합
Benchmark	ablation framework 존재, result schema 빈약	PARTIAL	telemetry/result v2 schema 먼저 완성
CI	unit/e2e regression 존재	EXISTING	schema/compat/cache/benchmark smoke gate 추가
Semantic summary cache	deterministic fact가 아닌 LLM 해석 재사용	REJECT	v2.0 자동 주입 금지

4.1 AS-IS 실행 비용 구조

AS-IS — correctness 절차와 분석 절차가 hot path에 함께 노출



그림 2. 현재 구조의 핵심 위험은 정확도 절차 자체가 아니라, 단순 문제에도 동일한 orchestration 비용이 앞단에서 발생할 수 있다는 점이다.

구조적 원인은 다음으로 분리한다.

- Orchestration tax: 문제 난이도와 무관하게 넓은 doctrine/workflow가 agent working set에 들어오기 쉽다.
- Context tax: raw decomp/full state/large tool output을 그대로 모델 context에 넣으면 `revq`의 본래 bounded-query 목적을 역행한다.
- Recomputation: 동일 분석을 다른 CLI/cache layer에서 다시 수행할 수 있다.
- Policy duplication: legacy/new workflow가 공존해 cache/verification/telemetry semantics가 경로별로 달라질 수 있다.
- Measurement gap: dev benchmark scaffold는 있으나 result contract가 약해 “왜 빨라졌는지”를 attribution하기 어렵다.

5. v2 아키텍처 원칙

ID	원칙	정의
A1	Query First	모델이 파일/로그 전체를 읽는 대신 deterministic artifact DB에 bounded query를 한다.
A2	FAST Default	정적 근거와 executable oracle이 명확하면 FAST를 유지한다.
A3	DEEP Conditional	복수 가설, 환경 민감성, 난독화/heap/kernel, oracle 모호성, repeated failure 때만 state/primitive/fanout/skeptic을 강화한다.
A4	Verification Invariant	FAST/DEEP과 무관하게 SOLVED는 deterministic evidence가 필요하다.
A5	Canonical Path	one concept → one implementation → thin compatibility adapters.
A6	Artifact Truth	repository/artifact/state가 truth이며 context는 projection이다.
A7	Cache Deterministic Facts	binary/tool/params/dependencies/schema/policy가 일치하는 deterministic 결과만 자동 재사용.
A8	Measurement/Ablation	변경 전 baseline, 한 번에 한 변경, cold/warm 분리, 회귀와 성능을 별도 판정.
A9	Budget Every Output	query/state/tool output에 byte/token budget과 truncation semantics를 둔다.
A10	No Feature Inflation	benchmark 병목이 아닌 신규 DB/cache/decompiler/full taint/generic auto-exploit은 거부한다.

6. TO-BE — Query-First Solver Runtime

Target Runtime — FAST 기본, DEEP 조건부 승격

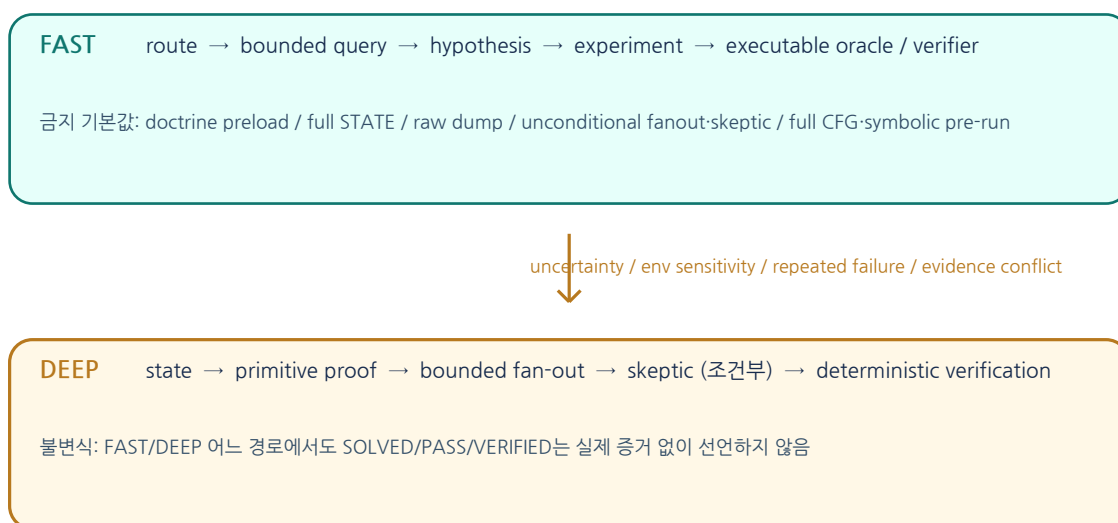


그림 3. FAST/DEEP 두 lane은 “정확도 vs 속도”가 아니라 “필요한 분석 엄격도”를 구분한다. 검증은 양쪽 모두 강제된다.

6.1 구성 요소

컴포넌트	책임	입력	출력/계약
Tiny Agent Contract	ROE, completion contract, FAST/DEEP, bounded query 원칙	stable instructions	작고 안정된 prefix
`rat route`	artifact/capability/signals 기반 deterministic routing	binary + environment	route result v1 + one skill + next query
Operator Skill	signal→first action→pivot→escalate→verify	route id	작은 route-local 지침
Query API	profile/function/oracle/slice/dyn을 bounded 결과로 제공	artifact ids / target	typed query result + artifacts
Artifact/Cache	deterministic 결과 저장·재사용	digest/key/provenance	immutable artifact + cache index
Model Reasoning Loop	가설 생성·실험 선택	bounded facts + state compact	hypothesis/next probe
Verifier	실행/oracle/schema/test로 성공 판정	candidate + environment	PASS/FAIL evidence
DEEP Runtime	state/primitive/fanout/skeptic 조건부 활성화	escalation reason	stronger evidence / reroute

6.2 Context layout

[STABLE PREFIX]

ROE / completion contract / FAST-DEEP rules / query semantics

[STABLE PER ROUTE]

exactly one selected operator skill

[DYNAMIC WORKING SET]

challenge profile

Function Card(s)

state compact --max-bytes N

new tool/query results

[NOT IN WORKING SET BY DEFAULT]

full doctrine / full STATE history / raw full decompile / repeated identical results

이 layout은 특정 모델 운영 전략이 아니라 context efficiency와 cache-friendly stable prefix를 위한 runtime contract다.

7. P0-0 — Measurement First

GAP

dev에는 ablation 뼈대가 있으나 benchmark result v1 schema가 `schema` 외 필드를 강제하지 않는다. 따라서 architecture 변경 전에 baseline을 고정할 수 있는 telemetry contract를 먼저 만든다.

7.1 Benchmark result v2 필수 필드

그룹	필드	목적
correctness	verified_solve, false_solved, oracle_pass	결과 정확도
latency	time_to_first_query, time_to_first_hypothesis, time_to_first_valid_primitive, time_to_verified_solve	solve latency 분해
context	input_tokens, output_tokens, peak_context_tokens, tool_output_bytes	context 압력
tools	tool_calls, duplicate_tool_calls, ghidra_runs, cfgfast_runs, symbolic_runs, subagent_count	orchestration/recomputation
cache	cache_requests, cache_hits, cache_hit_ratio, bytes_reused, cold_warm	cache 효과
reasoning	hypotheses_created, hypotheses_refuted, pivot_count, deep_escalations	reasoning 경로 변화
artifacts	functions_decompiled, raw_output_size, compressed_output_size, artifact_count	query 압축 효과

7.2 `duplicate\_tool\_calls` 정의

단순 command string 중복이 아니라 canonical operation fingerprint 중복으로 집계한다. 동일 artifact digest + tool build + normalized parameters + dependency set + output schema가 같은 호출을 중복 후보로 본다. cache hit은 중복 “실행”으로 세지 않되 `cache\_requests`에는 포함한다.

```
operation_fingerprint = sha256({
  "tool": {"name": ..., "build_digest": ...},
  "inputs": sorted(input_digests),
  "parameters": normalized_parameters,
  "dependencies": dependency_versions,
  "policy_digest": ...,
  "output_schema": ...
})
```

7.3 변경 파일

- `bin/ratlib/telemetry.py` — event/result aggregation, operation fingerprint, cache counters.
- `benchmarks/schemas/rat.benchmark-result.v2.json` — required fields와 단위 정의.
- `benchmarks/ablations/V2-A\*.yaml` — architecture ablation. 기존 A0~A5는 legacy baseline reference로 보존.
- `tests/test\_telemetry.py` / benchmark schema validation test — duplicate/cold-warm/failure path 포함.

7.4 Acceptance

- 행동 변화 없이 현행 A0 baseline이 기록 가능하다.
- 동일 분석의 repeated execution과 cache hit이 구분된다.
- timeout/partial/truncated run은 성공 cache hit로 계산되지 않는다.
- metric 단위(ms, bytes, tokens, count)가 schema에 고정된다.

8. P0-1/P0-2 — Slim Agent Runtime + FAST/DEEP

VERIFIED: `CLAUDE.md`는 START HERE에서 doctrine 문서들을 순서대로 읽도록 하고, Scout/fan-out/primitive/skeptic을 폭넓게 노출한다. [R1]

PROPOSED: root contract는 약 1~2KB의 안정된 규칙만 남기고, 세부 doctrine/knowledge는 route에 따라 lazy load한다.

8.1 Root contract에 남길 7개 규칙

- Authorized CTF/local artifact ROE와 금지 행위.
- 목표는 실제 verifier/flag까지이며 성공 주장은 증거가 필요.
- 첫 동작은 `rat route`.
- route가 선택한 operator skill 하나만 기본 로드.
- raw dump보다 bounded query 사용.
- 불확실/환경 민감/repeated failure/evidence conflict면 DEEP 승격.
- `SOLVED`/`PASS`는 deterministic verification 없이는 금지.

8.2 FAST에서 기본 비활성

기능	FAST 기본값	DEEP 조건
Full doctrine preload	OFF	route skill로 해결 불가할 때 관련 문서만
Full STATE history	OFF	항상 compact projection 우선
Raw Ghidra dump	OFF	Function Card/target decomp가 부족할 때
Fan-out	OFF	서로 다른 유효 가설 ≥ 2
Skeptic	OFF for clear executable oracle	remote/env-sensitive/ambiguous verifier
Full CFG/symbolic pre-run	OFF	oracle/path 조건이 확인된 뒤 필요할 때
Scout subagent	OFF	bounded query budget을 넘는 조사만

8.3 Progress Novelty Governor

시간 기반 “stuck” 판정보다 새 artifact/fact/hypothesis elimination/primitive advancement가 있는지를 본다. 기본안: 최근 5회 tool/query 호출에서 새 artifact digest, finding revision, ruled-out route, primitive status change가 하나도 없으면 강제 `re-route` 또는 DEEP escalation reason을 기록한다.

9. P0-3 — Canonical Cache / Artifact

VERIFIED: `ratlib/cache.py`는 이미 content-addressed key를 제공한다. `revq`는 `revq.json`, `decomp`는 `decomp`와 별도 provenance cache를 사용한다. `ratlib.analysis`의 envelope는 cache key=None/hit=False다. [R2,R3,R5,R7,R8]

INFERRED: “캐시가 없음”이 아니라 캐시 namespace와 lifecycle이 분산된 상태다. 새 cache subsystem은 중복을 늘린다.

9.1 Canonical key

```
rat.cache-key/v2 = sha256(canonical_json({
  artifact_inputs: [{role, digest, size}],
  tool: {name, version, build_digest},
  parameters: normalized,
  dependencies: exact_versions_or_digests,
  policy_digest: ...,
  output_schema: ...,
  analysis_schema_version: ...
}))
```

9.2 Migration model

현재	v2 canonical 처리	호환
`.revq.json`	revmap artifact로 ingest + ratlib index key 생성	기존 sidecar read adapter 한 release 유지
`.decomp/`	함수 decomp artifacts + provenance key를 ratlib index로 연결	기존 directory export는 adapter로 읽기
`.rat-profile/slice/dyn`	envelope 생성 시 canonical cache key 계산	기존 CLI output shape 유지
legacy deterministic tools	`ratlib.contracts.execute()` adapter 계속 사용	stdout/stderr artifacts 유지

9.3 Cache hit envelope fix

현재 legacy adapter는 cache hit 시 저장된 이전 envelope를 그대로 반환할 수 있어 provenance의 `cache.hit=false`와 이전 invocation_id가 남을 수 있다. v2에서는 artifact는 재사용하되 invocation envelope는 새로 생성한다.

```
{
  "invocation_id": "invoke_new",
  "status": "ok",
  "artifacts": ["reused immutable refs"],
  "provenance": {
    "cache": {
      "key": "sha256:...",
      "hit": true,
      "source_invocation": "invoke_old"
    }
  }
}
```

9.4 자동 재사용 허용/금지

분류	자동 cache	비고
ELF/file profile, symbols/imports, strings/xrefs, CFG, function boundary	YES	tool/version/params/dependencies/schema key 필요
Ghidra decompile, ROP gadgets, deterministic Function Card facts	YES	partial/truncated/stale는 제외
Dynamic trace	조건부	scenario/environment digest가 key에 포함될 때만
LLM semantic summary / hypothesis / prose interpretation	NO (v2.0)	deterministic fact 아님; 자동 context injection 금지

10. P0-4 — Context Projection: `state compact` 강화

VERIFIED: `state delta`, `state compact`, `state checkpoint`가 이미 존재하므로 새 `state snapshot` subsystem은 중복이다. [R6]

PROPOSED: `state compact`를 bounded projection API로 확장한다. append-only STATE/event store는 그대로 truth source로 유지한다.

```
state compact \
  --max-bytes 6000 \
  --format text \
  --include active,confirmed,next \
  --recent-ruled-out 5
```

10.1 우선순위 projection

순위	내용	초과 시
1	invalidating alerts / evidence invalidation	절대 유지
2	confirmed facts/findings	절대 유지
3	PASS primitives + environment/input digest	절대 유지
4	active hypotheses	낮은 우선순위부터 drop
5	next probes	상위 N 유지
6	recent ruled-out routes	최근/관련 항목만
7	notes/old history	기본 drop

10.2 Context budget semantics

- `truncated=true`, `omitted\_counts`, `cursor`를 출력해 projection이 완전한 history가 아님을 명시.
- JSON과 text 두 format 제공. text는 모델 working set, JSON은 deterministic downstream/benchmark용.
- 동일 state cursor + policy + max-bytes이면 deterministic output이 되도록 정렬 규칙 고정.
- projection이 evidence lifecycle을 변경하지 않는다.

11. P1 — Single Front Door: `rat`

새 분석 엔진이 아니라 thin dispatcher를 만든다. 기존 binaries는 compatibility adapter로 유지하고 새 문서/agent contract는 `rat`만 canonical path로 사용한다.

```
rat route ./chall
rat query func check_flag
rat query oracle [check_flag]
rat query slice --backward 0x401620 --source stdin --depth 2
rat dyn --scenario run.json
rat verify --scenario verify.json
rat state compact --max-bytes 6000
rat cache stats
```

11.1 `rat route` contract

`rat-doctor` capability + `rat-profile` + `revq --fast`의 deterministic 신호를 조합한다. 라우터 자체가 “풀이”를 하지 않으며 분류 근거를 모두 노출한다.

```
{
  "schema": "rat.route-result/v1",
  "track": "rev",
  "subroute": "rev-checker",
  "confidence": 0.91,
  "signals": [
    {"kind": "import", "value": "memcmp", "quality": "direct"},
    {"kind": "string", "value": "Correct!", "quality": "direct"}
  ],
  "capabilities": {"native": true, "angr": true, "ghidra": true},
  "skill": "skills/rev-checker/SKILL.md",
  "next": [{"query": "func", "target": "check_flag"}]
}
```

11.2 Router invariants

- confidence는 heuristic이며 fact와 분리.
- 하나의 기본 skill만 반환. 복수 후보는 alternatives로 명시하되 자동 동화 로드 금지.
- tool/capability missing은 route 실패가 아니라 degradation reason으로 구조화.

- route result는 artifact digest와 source signal을 가져 재현 가능해야 함.

12. P1 — Operator Skill Layer

기존 `knowledge/`를 다시 쓰지 않는다. knowledge는 reference layer로 남기고, hot path에는 작고 행동 지향적인 operator skill만 둔다.

```
skills/
├── rev-checker/SKILL.md
├── rev-vm/SKILL.md
├── rev-packed/SKILL.md
├── rev-symbolic/SKILL.md
├── pwn-stack/SKILL.md
├── pwn-format/SKILL.md
├── pwn-heap/SKILL.md
├── pwn-rop/SKILL.md
└── pwn-kernel/SKILL.md
```

각 skill의 허용 섹션은 다음으로 제한한다.

섹션	내용
SIGNALS	이 skill을 선택할 deterministic/heuristic 신호
FIRST ACTION	첫 bounded query/experiment
PIVOT	결과별 다음 행동
ESCALATE	DEEP 승격 조건
VERIFY	성공으로 인정하는 deterministic evidence

Skill은 doctrine 복제본이 아니며, 1개 파일이 지나치게 커지면 route 분류가 잘못 설계된 것으로 본다.

13. P1 — Function Card v2

VERIFIED: `revq --func`는 이미 함수 주소·크기·block/instruction 수·callers·calls·strings를 bounded card로 출력하고, 본문은 `decomp`로 넘긴다. [R2]

PROPOSED: 별도 `rat-function-card`를 만들지 않고 revmap/function-card schema를 확장한다.

```
{
  "schema": "rat.function-card/v2",
  "function": { "name": "check_flag", "address": "0x1450", "size": 231 },
  "facts": {
    "callers": [ "main" ],
    "callees": [ "transform", "memcmp" ],
    "strings": [ "Correct!", "Wrong!" ],
    "input_apis": [ "read" ],
    "compare_sites": [ { "address": "0x1594", "api": "memcmp", "length": 32 } ],
    "oracle_candidates": [
      { "kind": "success", "xref": "0x1608", "string": "Correct!" },
      { "kind": "failure", "xref": "0x1630", "string": "Wrong!" }
    ]
  },
  "heuristics": { "role": "checker", "score": 8, "reasons": [ "..."] },
  "unresolved": { "indirect_calls": 0, "truncated": false },
  "next": [ { "query": "oracle", "target": "check_flag" } ]
}
```

13.1 Schema rule

- `facts`는 deterministic extractor 결과만.
- `heuristics`는 score/reasons와 detector version을 포함.
- `unresolved`는 alias/indirect/truncation을 숨기지 않음.

- LLM prose summary는 기본 card에 넣지 않음.
- card 생성에 사용한 revmap/decomp digest를 provenance로 연결.

14. P1 — Oracle Detector → Symbolic → Concrete Verify

VERIFIED: `revq`에는 Correct/Wrong/Success/Invalid/flag 계열 interesting string, xref, compare/import 정보와 interesting-function heuristic이 이미 있다. `CLAUDE.md`도 symbolic result를 real binary로 concrete verify하도록 요구한다. [R1,R2]

GAP: 이 정보가 success/fail basic block과 symbolic find/avoid에 자동 연결되는 canonical query contract는 없다.

Oracle wiring — 새 분석 엔진보다 기존 결과 연결



그림 4. oracle wiring은 주소 복붙과 반복 분석을 제거한다.

14.1 자동 연결 조건

- success/failure 후보가 각각 단일·명확하고 xref가 concrete basic block으로 resolve될 때만 symbolic hint 자동 생성.
- 후보가 여러 개면 `ambiguous=true`와 ranking reasons를 반환하고 자동 실행하지 않음.
- symbolic candidate는 성공 증거가 아님. candidate input을 실제 binary/scenario로 실행해 oracle을 확인해야 PASS.
- local verify가 remote equivalence를 증명하지 않는 기존 limitation 유지.

15. P1/P2 — Bounded Backward Data Slice

GAP

`rat-slice`는 현재 angr CFGFast callgraph 경로와 VEX block metadata를 제공하지만 source→sink value-flow proof가 아니다. 이는 소스에 limitation으로 명시되어 있다. [R5]

15.1 MVP 범위

```

rat query slice \
  --mode data \
  --backward 0x401620 \
  --source stdin \
  --depth 2
  
```

기능	MVP	비고
within-function def/use	YES	register/VEX temp/local stack 기준
register dependencies	YES	SSA/IR 기반 bounded tracking
stack-local dependencies	YES	확정 가능한 offset만
direct-call summary	YES	known ABI/callee summary
interprocedural	depth ≤ 2	budget 강제
heap/global alias full resolution	NO	unresolved로 보고

기능	MVP	비고
indirect call complete resolution	NO	unresolved edge 기록
full-program taint	REJECT v2.0	복잡도/비용 대비 근거 없음

15.2 결과 계약

```
SUCCESS@0x401620
↑ branch@0x4015fa
↑ memcmp.result
↑ tmp[0:32]
↑ transform(input)
↑ read(stdin, input, 32)
```

```
coverage: bounded-def-use + direct-call-summary depth=2
unresolved_aliases: 1
unresolved_indirect_calls: 0
claim: dependency-candidate (NOT full proof)
```

미해결 alias/indirect가 있으면 결과를 “proof”로 승격하지 않는다. 이 제한 자체가 schema와 diagnostics에 포함되어야 한다.

16. P2 / v2.1 — Benchmark-Driven Extensions

아래 기능은 유용할 가능성이 있지만 v2.0 기본 구현 대상이 아니다. telemetry에서 반복 병목이 확인될 때만 gate를 통과한다.

후보	추가 조건	권장 구현 형태	우선순위
Differential trace	byte-wise checker/state machine/VM이 TTF 병목	새 CLI 대신 `rat-dyn --compare A B`	P2
Constraint extractor	compare/length/transform 수동 추출이 반복 병목	Function Card compare facts 확장부터 시작	P2
syscall/seccomp observer	PWN corpus에서 seccomp/syscall uncertainty가 반복	rat-dyn/runtime extension	P2
allocator event collector	heap corpus에서 allocator state 관측 비용이 반복	bounded event artifact	P2
coverage fuzz adapter	input discovery가 dominant bottleneck	기존 runner/artifact에 adapter	P2
rr/replay	nondeterministic debugging이 dominant bottleneck	optional backend	P2
LLM semantic cache	정확도 손실 없이 이득이 실측될 때	격리된 interpretation artifact, auto-inject 금지	REJECT v2.0

17. 핵심 인터페이스 / 스키마 설계

17.1 Query result envelope

```
{
  "schema": "rat.query-result/v1",
  "query": {"kind": "func", "target": "check_flag", "budget_bytes": 8192},
  "status": "ok|partial|error",
  "facts": [...],
  "heuristics": [...],
  "artifacts": [{"kind": "function-card", "digest": "sha256:..."}],
  "coverage": {"complete": false, "scope": "...", "omitted": 3},
  "diagnostics": [...],
  "provenance": {
    "inputs": [...],
    "tool": {...},
    "dependencies": {...},
    "cache": {"key": "sha256:...", "hit": true, "source_invocation": "..."}
  }
}
```

17.2 Output budget contract

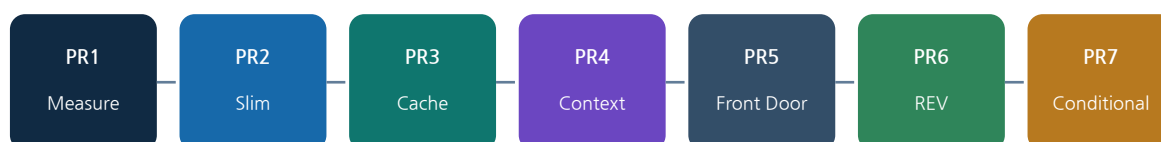
- 모든 query는 `budget\_bytes` 또는 equivalent limit을 가진다.
- limit을 초과하면 임의 문자열 절단보다 item boundary에서 잘라 `truncated=true`, `omitted\_count` 제공.
- 원문 전체가 필요하면 artifact digest를 반환하고 context에는 넣지 않는다.
- partial/timeout/dependency_missing은 status/diagnostics로 구분한다.

17.3 Error taxonomy

코드	의미	재시도 정책
input_invalid	binary/scenario/digest 불일치	수정 전 재시도 금지
dependency_missing	Ghidra/angr/gdb 등 없음	capability degradation 또는 install
timeout	budget 초과	동일 결과 cache 금지; budget/target 축소
partial	분석 일부만 유효	coverage를 명시하고 downstream claim 제한
stale_cache	provenance 불일치	recompute
ambiguous	복수 후보	사람/모델 hypothesis 필요
verification_fail	candidate가 oracle 통과 실패	SOLVED 금지, hypothesis refute

18. 구현 계획 — PR 단위

Implementation Sequence



원칙: 한 PR당 한 가설을 검증하고, 동일 corpus / environment / timeout에서 회귀·성능을 분리 측정한다.

그림 5. 한 PR에서 여러 architecture 가설을 동시에 바꾸지 않는다.

PR	Goal	Files/범위	Benchmark	Acceptance
PR1 Measurement First	행동 변경 없이 baseline 계측	`ratlib/telemetry.py`, benchmark result v2 schema, V2 ablation files, tests	현행 corpus를 cold/warm으로 기록; duplicate/cache/context metrics	모든 필수 field와 단위가 schema validation 통과
PR2 Slim Runtime	entrypoint/context orchestration tax 축소	`CLAUDE.md`/`AGENTS.md`, route skill loading rules, progress governor	A0 vs slim only	solve rate 비하 없음; context/tool-call 감소 추세
PR3 Canonical Cache	분산 cache를 ratlib/artifact index로 통합	`ratlib/cache.py`, `contracts.py`, `analysis.py`, revq/decomp adapters	cold vs warm + stale/tool-version cases	hit provenance 정확, partial/truncated 미재사용
PR4 Context Projection	STATE/raw output working set 축소	`state compact --max-bytes`, Function Card v2, query budget	peak context / output bytes	truth history 손실 없이 projection bounded
PR5 Query Front Door	one canonical UX	`bin/rat` dispatcher, route/query schema, compatibility adapters	CLI golden/e2e	legacy 명령과 결과 의미 호환
PR6 REV Acceleration	주소 연결 및 data dependency 조사 비용 축소	oracle detector/wiring, slice data mode	REV checker corpus	oracle candidate concrete verify, slice limitation 명확
PR7 Conditional	실측 병목만 확장	dyn compare / constraints / pwn observers 중 증명된 것만	해당 병목 corpus	추가 기능의 독립적인 ablation 이득

19. 테스트 / 회귀 설계

19.1 Unit tests

대상	검증
Cache key determinism	input ordering, parameter normalization, version/schema/policy 변화 시 key 변화.
Cache hit envelope	새 invocation_id + hit=true + source_invocation 연결; artifact digest 동일.
State compact budget	동일 state/cursor에서 deterministic projection; max-bytes 초과 시 priority 유지.
Route rules	동일 profile/signals에 동일 subroute/skill; missing capability degradation.
Function Card	facts/heuristics/unresolved 분리; compare/oracle candidate detector false-positive fixtures.
Data slice	known def-use fixture, alias unresolved fixture, depth budget, indirect call partial semantics.
Telemetry	duplicate fingerprint, cold/warm cache, timeout/partial/truncated accounting.

19.2 Integration / E2E

- `rat route` → query func → query oracle → verify` synthetic checker fixture.
- `rat route` → query slice --mode data` source-to-compare fixture; unresolved alias fixture도 포함.
- legacy `revq/decomp/state/rat-\*` 명령과 canonical `rat` adapter의 의미 호환성.
- cache warm run에서 Ghidra/CFGFast 재실행이 실제로 줄어드는지 tool telemetry 확인.
- cache stale: binary/tool/script/dependency/schema 변경 시 miss 후 recompute.
- CI의 기존 unittest/e2e_artifact_handoff/e2e_analysis/e2e_orchestration/pkselftest 유지.

19.3 Failure-path tests

- Ghidra/angr/gdb dependency missing
- tool timeout
- output truncation

- partial Ghidra export
- STATE evidence invalidation after compact
- ambiguous oracle
- symbolic candidate concrete verification FAIL
- cache sqlite corruption/read failure fallback

20. Benchmark / Ablation 방법

성능 개선은 “느낌”이 아니라 동일 조건 비교로 판단한다. 모델 운영 설정은 이 문서 범위에서 제외하지만, benchmark에서는 모델/환경/timeout/tool versions를 고정 변수로 취급해야 architecture attribution이 가능하다.

20.1 권장 ablation

Ablation	변경	검증 질문
V2-A0	현재 baseline + telemetry only	측정 자체가 행동을 바꾸지 않는가?
V2-A1	+ slim endpoint/FAST-DEEP	context/tool-call 감소가 solve-rate를 해치지 않는가?
V2-A2	+ canonical cache	warm wall-time/recompute 감소가 실제인가?
V2-A3	+ state compact/function card	peak context와 raw output이 감소하는가?
V2-A4	+ front door/router	duplicate path와 route latency가 감소하는가?
V2-A5	+ oracle wiring	REV time-to-first-valid-candidate가 감소하는가?
V2-A6	+ backward data slice	해당 corpus에서 decompile/raw analysis 호출이 감소하는가?

20.2 실행 규칙

- 동일 challenge corpus, toolchain, environment, resource limit, timeout.
- 각 ablation cold 3회 + warm 3회 권장. 기존 dev A5의 cold/warm 개념을 확장.
- REV/PWN/난이도 층화. 쉬운 checker에서 unnecessary DEEP 비율 별도 집계.
- outlier를 숨기지 않고 median + p90 + failure count 보고.
- verified_solve가 하락한 실험은 토큰/속도가 좋아도 채택하지 않음.
- PR별로 하나의 주요 가설만 켜서 attribution 가능하게 함.

21. v2 Release Gate

PROPOSED 아래 수치는 측정값이 아니라 초기 release target이다. PR1 baseline 측정 후 corpus 분포에 맞게 조정해야 하며, 달성했다고 현재 주장하지 않는다.		
지표	초기 target	판정
false SOLVED	0	하드 게이트
verified solve rate	$\geq$ v1 baseline	하드 게이트
median time-to-verified-solve	$\leq$ 70% of v1	성능 target
median peak context	$\leq$ 55% of v1	효율 target
tokens / verified solve	$\leq$ 65% of v1	효율 target
duplicate tool executions	$\leq$ 25% of v1	orchestration target
warm deterministic cache hit ratio	$\geq$ 70%	cache target
easy REV unnecessary DEEP rate	$\leq$ 10%	routing target

Release는 implementation + tests + regression + relevant benchmark + compatibility review + documentation sync + reproducible evidence가 모두 갖춰져야 한다.

22. Compatibility / Migration / Rollback

22.1 Canonicalization 단계

단계	이름	내용
1	사용처 계측	legacy/new CLI 및 cache read/write 횟수를 telemetry로 확인.
2	canonical 결정	`rat` + `ratlib.cache.artifact` + typed query schema를 공식 path로.
3	thin adapters	기존 revq/decomp/rat-* binary는 canonical implementation을 호출하거나 결과를 변환.
4	dual-read / single-write	migration 동안 legacy cache read는 허용하되 새 write는 canonical index로.
5	compat test	CLI output/exit code/schema compatibility test.
6	deprecation telemetry	legacy 직접 사용이 0에 근접했을 때 removal plan.

22.2 Rollback

- 각 PR은 feature flag/ablation switch로 독립 rollback 가능해야 함.
- cache migration은 immutable artifacts를 삭제하지 않고 index adapter만 되돌릴 수 있어야 함.
- STATE schema/event stream은 파괴적 migration 금지; compact는 view-only change.
- front door 도입 중 legacy CLI를 제거하지 않음.
- release gate 실패 시 마지막 통과 ablation 조합으로 복귀.

23. 위험 분석

위험	영향	완화	등급
Slim endpoint가 중요한 doctrine을 누락	solve-rate/false-SOLVED 약화	deterministic verifier hard gate 유지, route-specific skill/DEEP escalation, regression corpus	High
Router 오분류	잘못된 skill/첫 query	signals+confidence+alternatives 노출, easy reroute, no auto irreversible action	Med
Cache stale/poison	오래된 분석 재사용	content/provenance key, partial/truncated 금지, schema/policy versioning	High
Function Card 과압축	중요한 control/data fact 누락	coverage/unresolved/truncated 명시, artifact fallback, benchmark	Med
Data slice 과신	false dependency proof	claim vocabulary 제한, unresolved alias/indirect 표기, concrete verification	High
Telemetry overhead	wall-time 왜곡	append-only lightweight events, A0 telemetry overhead 자체 측정	Low
Legacy compatibility 장기화	중복 경로 고착	deprecation telemetry + single-write + removal gate	Med
Semantic cache 재도입	오판 고속 재주입	v2.0 reject, 재검토 시 interpretation/unverified 격리	High

24. 보안 및 검증 경계

v2 architecture는 CTF용 로컬 분석 성능을 높이되, 저장소가 가진 범위-검증 불변식을 약화하지 않는다. 구현/benchmark 기본 대상은 repository fixtures, benchmark corpus, 제공 challenge artifact, local process, Docker/VM/loopback이다.

항목	정책
SOLVED/PASS	실제 binary execution/oracle/test/schema/evidence 없이는 선언 금지
Remote equivalence	local PASS가 remote equivalent를 자동 의미하지 않음
Analysis output	heuristic/result artifact 자체는 primitive proof가 아님

항목	정책
Scope	제공 artifact-local environment 중심; 제3자 스캔/credential/persistence/stealth/DoS 기능 추가 금지
Caching	semantic interpretation을 deterministic fact로 승격 금지
Failure reporting	partial/timeout/unresolved를 성공처럼 normalize 금지

25. P0 / P1 / P2 / Reject

우선순위	항목	근거
P0	Telemetry/result v2 + baseline	측정 없이는 변경 효과 attribution 불가
P0	Slim root contract + FAST/DEEP	orchestration/context tax 직접 감소
P0	Canonical cache/artifact	중복 recomputation과 cache 세게 분산 해소
P0	state compact budget projection	새 state subsystem 없이 context working set 축소
P1	`rat` front door/router	canonical UX와 single workflow
P1	operator skills	knowledge preload를 signal→action 단위로 축소
P1	Function Card v2	기존 revq card를 structured bounded facts로 승격
P1	Oracle wiring	기존 interesting/xref/symbolic/verify를 연결
P1/P2	Backward data slice MVP	현재 call-path-only GAP 보완
P2	dyn compare / constraint extractor / PWN observers	benchmark 병목 확인 후
REJECT	새 state DB/cache/decompiler	중복 subsystem
REJECT	full taint first / generic auto-exploit	복잡도·검증 비용 대비 근거 부족
REJECT	semantic cache auto-injection	deterministic truth와 충돌
REJECT	unconditional fanout/skeptic/full Ghidra/CFG	쉬운 문제 hot-path 비용 증가

26. Definition of Done

CTF-Rat v2는 “새 CLI가 동작함”으로 완료되지 않는다. 아래가 모두 충족되어야 한다.

- Implementation: canonical interfaces와 adapters가 구현됨.
- Tests: unit + integration + failure path + compatibility test GREEN.
- Regression: 기존 CI와 relevant selftest GREEN.
- Benchmark: baseline 대비 solve-rate/TTF/context/cache/tool-call 수치가 재현됨.
- Compatibility: legacy 직접 사용처와 migration/rollback이 확인됨.
- Documentation: AGENTS/CLAUDE, CLI help, schema/reference 동기화.
- Evidence: commit/tool versions/environment/corpus/timeout/reproduction log를 보존.
- No false completion: 측정하지 않은 항목은 MEASURED/VERIFIED로 표시하지 않음.

최종 기준

기능 수가 늘었는가?보다 더 적은 context·중복·wall-clock으로 동일하거나 더 높은 verified solve rate를 달성했는가?를 v2 성공 기준으로 삼는다.

부록 A. 파일 변경 맵

파일/영역	조치	내용
`CLAUDE.md` / `AGENTS.md`	MODIFY	tiny stable contract, FAST/DEEP, `rat route` first, verifier invariant
`bin/rat`	ADD	thin dispatcher / route / query front door
`bin/revq`	MODIFY	Function Card v2, canonical artifact/cache adapter; 기존 command 호환
`bin/decomp` + `ratlib/decomp_cache.py`	MODIFY/ADAPT	canonical cache index 연동; legacy sidecar read compat
`bin/state`	MODIFY	compact --max-bytes/--format/priority projection
`bin/ratlib/cache.py`	MODIFY	v2 key/version, stats, provenance helpers
`bin/ratlib/contracts.py`	MODIFY	cache-hit new envelope, telemetry hook
`bin/ratlib/analysis.py`	MODIFY	analysis cache wiring, slice data mode, structured coverage
`bin/ratlib/telemetry.py`	ADD	benchmark events/metrics/fingerprint aggregation
`skills/*/SKILL.md`	ADD	small operator skills
`benchmarks/schemas/rat.benchmark-result.v2.json`	ADD	required telemetry contract
`benchmarks/ablations/V2-A*.yaml`	ADD	stepwise v2 ablations
`tests/*`	MODIFY/ADD	cache/state/router/query/slice/telemetry/compat/failure tests
`.github/workflows/regression.yml`	MODIFY	new schema/unit/e2e smoke without breaking existing suite

부록 B. Architecture Decision Records

ID	결정	상태	이유
ADR-001	FAST default, DEEP conditional	Accepted	모든 문제에 동일 orchestration을 적용하지 않는다.
ADR-002	ratlib cache/artifact canonical	Accepted	새 cache subsystem을 만들지 않는다.
ADR-003	state compact 강화	Accepted	새 state snapshot/DB를 만들지 않는다.
ADR-004	revq Function Card v2	Accepted	새 function-card CLI를 만들지 않는다.
ADR-005	semantic cache auto injection	Rejected v2.0	interpretation을 fact처럼 재사용할 위험.
ADR-006	full-program taint first	Rejected	MVP bounded slice로 실제 병목을 먼저 검증.
ADR-007	single front door `rat`	Accepted	legacy binaries는 thin compatibility adapters.
ADR-008	verification hard invariant	Accepted	FAST도 verifier를 우회하지 않는다.

부록 C. 예시 운영 흐름

C.1 단순 REV checker — FAST

```
$ rat route ./chall
TRACK rev / SUBROUTE rev-checker
NEXT rat query func check_flag

$ rat query func check_flag
COMPARE memcmp(...)
ORACLE success/fail candidates
NEXT rat query oracle check_flag

$ rat query oracle check_flag
SUCCESS 0x401620 / FAIL 0x401648
SYMBOLIC_HINT find=0x401620 avoid=0x401648

$ rat verify --candidate candidate.bin --scenario verify.json
PASS executable oracle
# DONE: no mandatory fanout/skeptic if evidence contract is satisfied
```

C.2 불확실한 REV — DEEP escalation

```
route → function card → oracle ambiguous
└─ two plausible checker paths
└─ data slice contains unresolved indirect call

DEEP(reason=ambiguous_oracle + unresolved_dependency)
state compact
bounded fan-out (max 2 initially)
targeted dyn/decomp
primitive/evidence reconciliation
concrete verifier
```

C.3 Warm cache

```
cold:
rat route → revq/CFG/Ghidra compute → immutable artifacts → cache index

warm:
same binary/tool/params/dependencies/schema/policy
→ cache hit
→ NEW invocation envelope(hit=true, source_invocation=...)
→ SAME immutable artifact digests
→ no duplicate expensive execution
```


부록 D. Source Reference

ID	Source	Use
[U1]	pasted.txt	업로드 분석: orchestration/context/dual-path 병목, FAST/DEEP, cache, Function Card, REV 기능 아이디어.
[U2]	pasted2.txt.txt	업로드 검증 메모: measurement-first, semantic cache caution, verifier invariant, 신규 기능 축소.
[U3]	CTF-Rat v2 업그레이드 계획.txt	통합 계획: Query-First architecture, PR 순서, release gates, state compact 수정, Sol section 포함 원문. 본 PDF에서는 Sol 운영 전략 부분 제외.
[R1]	GitHub main/CLAUDE.md @ 01a1a9a...	agent entrypoint / doctrine/workflow / verification policy.
[R2]	GitHub main/bin/revq	bounded static extraction, xref/interesting/evasion, function card, sidecar cache.
[R3]	GitHub main/bin/ratlib/cache.py	canonical P1 content-addressed cache key/index.
[R4]	GitHub main/bin/ratlib/contracts.py	legacy deterministic tool envelope/artifact/cache adapter.
[R5]	GitHub main/bin/ratlib/analysis.py	bounded analysis executors, slice limitation, cache fields.
[R6]	GitHub main/bin/state	typed STATE commands: delta/compact/checkpoint.
[R7]	GitHub main/bin/decomp	Ghidra cache + function lookup/fallback.
[R8]	GitHub main/bin/ratlib/decomp_cache.py	Ghidra provenance key.
[R9]	GitHub main/.github/workflows/regression.yml	current regression CI.
[D1]	GitHub dev/benchmarks/schemas/rat.benchmark-result.v1.json	minimal benchmark result schema.
[D2]	GitHub dev/benchmarks/ablations/A0..A5.yaml	ablation scaffold and cold/warm concepts.

Repository: <https://github.com/b0nsu/CTF-Rat>

Source snapshot recorded on 2026-08-23. The repository may evolve after this design snapshot.

최종 요약

v2

CTF-Rat v2 = small stable instruction + single canonical workflow + deterministic analysis artifacts + bounded query interface + high cache reuse + measurement-driven development + conditional deep analysis + strict deterministic verification.

구현 우선순위는 다음 한 줄로 고정한다.

measure → slim → unify cache → compact context → query front door → oracle wiring → bounded data slice

이 설계서의 가장 중요한 비기능 요구사항은 “새 기능을 늘리기 전에 기존 deterministic 분석 결과를 더 작고, 재사용 가능하고, 연결된 형태로 제공하라”는 것이다.